

## 3.109 Experimenting with text generation

May 7, 2023

### 1 Conditional frequency distributions

#### 1.1 1. Using simple bigrams

##### 1.1.1 1.1 Download the Brown corpus

```
[1]: import nltk
      from nltk.corpus import brown
      nltk.download('brown')
```

[nltk\_data] Downloading package brown to /home/jovyan/nltk\_data...

[nltk\_data] Package brown is already up-to-date!

[1]: True

##### 1.1.2 1.2 Create a bigram model

```
[2]: text = brown.words(categories='news')
      bigrams = nltk.bigrams(text)
      cfd = nltk.ConditionalFreqDist(bigrams)
```

```
[3]: def generate_text(cfdist, word, num=50):
      for i in range(num):
          print(word, end=' ')
          word = cfdist[word].max()
```

##### 1.1.3 1.3 Test it

Use a variety of different words!

```
[4]: # here is just one example, try some others yourself
      generate_text(cfd, 'pass')
```

pass enabling legislation to the first time . The President Kennedy , and the first time . The President Kennedy , and the first time . The President Kennedy

, and the first time . The President Kennedy , and the first time . The President Kennedy , and the

## 1.2 2. Make it more generative

Pick the next word at random from the list of bigrams

1. Add a parameter for the number of bigrams to consider
2. Assign the bigrams for the current word to a frequency distribution
3. Create a list of the top N bigrams
4. Pick one at random and assign to the variable 'word'

```
[5]: # fill in the blanks marked X
# import random
# def generate_text(cfdist, word, num=100, X=X): #1
#     for i in range(num):
#         print(word, end=' ')
#         fdist = XXX[XXX] #2
#         words = XXX #3
#         word = random.XXX(XXX) #4
```

### 1.2.1 2.1 Reveal the solution (only if you get stuck!)

```
[6]: # solution
import random
def generate_text(cfdist, word, num=50, n=2):
    for i in range(num):
        print(word, end=' ')
        fdist = cfdist[word]
        words = list(fdist.keys())[:n]
        word = random.choice(words)
```

```
[7]: # solution
import random
def generate_text_no_random(cfdist, word, num=50, n=2):
    for i in range(num):
        print(word, end=' ')
        word = cfdist[word].max()
```

## 1.3 3. Test your solution using different values of N

```
[8]: generate_text(cfd, 'He', n=2)
```

He told the City of the election , `` deserves the election , which had been under fire for the City Executive Committee , `` deserves the election produced

```
`` deserves a relative merits of the City of Atlanta's new management got 402 .  
`` deserves a relative merits awe
```

```
[9]: generate_text(cfd, 'He', n=5)
```

```
He also . It urged that Fulton County purchasing departments which it said , but  
the City welfare consulting firm composed of the City judge excused one brief  
power which was received and thanks of such funds might give it added , but in  
the manner , but who has
```

```
[10]: generate_text_no_random(cfd, 'He', n=2)
```

```
He said . The President Kennedy , and the first time . The President Kennedy ,  
and the first time . The President Kennedy , and the first time . The President  
Kennedy , and the first time . The President Kennedy , and the first time . The  
President
```

#### 1.4 4. Discussion:

Why does the original version get stuck in a loop so easily?

How does introducing some randomness solve this problem?

What effect does increasing N (the number of bigrams to consider) have? Does it make the text more or less intelligible? If so, why?

```
[ ]:
```